Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 4

Requirements Validation, Stakeholders, Backlogs, And Planning Realism

Team Standup Meetings

When Software Projects Fail

- Mars Climate Orbiter
- Denver Airport Baggage System
- Healthcare.gov Launch
- Knight Capital Trading Loss

Did coding fail, or something earlier?

Requirements Validation = Risk Management

Validation Reduces:

- Technical Risk
- Schedule Risk
- Cost Risk
- Stakeholder Misalignment
- Rework

Professional Requirement Quality Framework

A High-Quality Requirement is:

- Atomic
- Unambiguous
- Verifiable
- Feasible
- Consistent
- Traceable
- Necessary
- Prioritized

Consider: “The system shall provide secure and fast login.”

Validation Is Collaborative

Stakeholder <----> Engineer

- Clarification
- Assumption testing
- Constraint discovery
- Trade-off discussion
- Negotiation

The Engineer’s Question Toolkit

- What problem are we solving?
- Who is the primary user?
- What are edge cases?
- What constraints exist?
- What does success look like?
- What happens if we don’t build this?

Refinement Changes Scope & Estimates

Before refinement:
“Prevent double booking” → 5 points
After refinement:

- Real-time locking
- Admin override
- Recurring reservations
- Conflict resolution UI

→ 13 points

(We’ll talk about “points” in a bit).

Stakeholder Role-Play Exercise

“Users must receive instant notifications via email, SMS, and push”

Clarify ➡️ Constrain ➡️ Negotiate

Professional Negotiation Strategies

Engineers negotiate by

- Framing trade-offs clearly
- Using estimates and data
- Offering phased solutions
- Protecting constraints
- Avoiding adversarial tone

❌ “We can’t do that.”

✓ “If we include SMS, timeline extends by 2 weeks…”

Requirements -> Backlog

Requirements

⬇️

User Stories

⬇️

Product Backlog

⬇️

Sprint Backlog

Story Points: What They Really Mean

Story Points Measure:

- Complexity
- Uncertainty
- Risk
- Relative Effort

NOT:

- Hours
- Days
- Individual Productivity

Why Not Linear Scaling?

If 1 = Trivial
And 10 = Extremely Complex,

Is 8 really just “a little more” than 7?

Why Fibonacci Scaling Works

1 – 2 – 3 – 5 – 8 – 13 – 21

Each step increases uncertainty tolerance.
As complexity increases, precision decreases.

What Happens at 13 and Above?

If a story is 13 or higher

- Too large
- Too uncertain
- Poorly understood
- Needs to be broken down

Example: Reservation System Stories

Consider these 4 user stories, and assign story points

- Display available rooms 2
- Create reservation 5
- Prevent double booking with conflict detection 8
- Add recurring reservations 13

Velocity and Forecasting

Velocity = 12 points per sprint
Backlog = 44 points
Forecast ≈ 3.7 sprints

Forecast ≠ Guarantee

From Backlog to Timeline Validation

Planning Steps

- Break stories into tasks
- Estimate duration
- Identify dependencies
- Build timeline
- Identify critical path

What the Gantt Reveals

Dependencies create bottlenecks
Critical path determines finish date

Points says “Yes” ✅
Timeline says “No” ❌

What is the Critical Path?

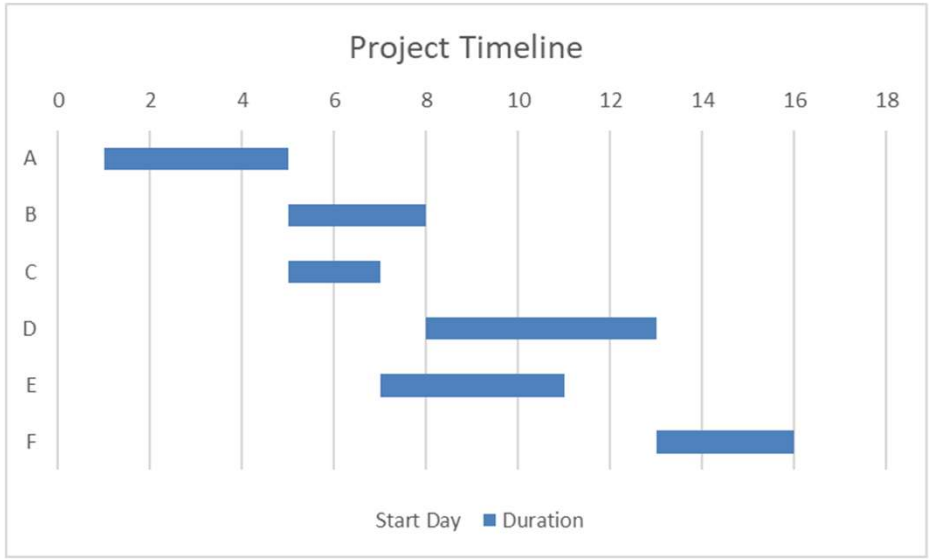
The Critical Path

- The longest chain of dependent tasks.
- Determines the minimum possible project duration.

Key Properties

- If any task on the critical path is delayed, the project is delayed
- Tasks not in the critical path have **slack**.

Finding the Critical Path

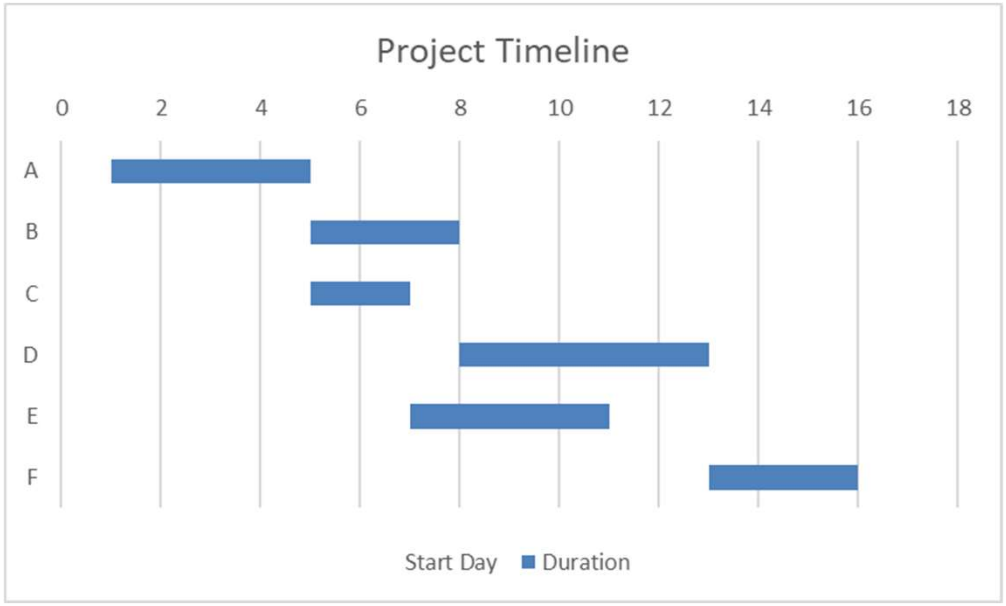


Step 1:
Identify Final Task (F)

Step 2:
Trace Dependencies Backward
F ⬅️ D ⬅️ B ⬅️ A

Critical Path:
A ➡️ B ➡️ D ➡️ F
= 15 days

What is NOT Critical?



Not on the critical path:

- Task C
- Task E

These tasks have **slack**.

Why This Matters

Critical Path = Schedule Risk

It helps you:

- Focus monitoring
- Prioritize resources
- Negotiate scope intelligently

Data-Driven Negotiation

Engineer says:
“If we include advanced conflict detection and email integration, the timeline exceeds our deadline.”

Options:

- Remove email from MVP
- Simplify conflict detection
- Phase admin features
- Extend timeline

Backlog + Gantt: Complementary Tools

Backlog answers “What should we build?”
Gantt answers “Can we build it in time?”

Validation requires both.

Final Takeaways

A requirement is not complete until

- It is validated
- It is refined with stakeholders
- It is prioritized
- It is estimated
- It fits the schedule

Engineering = disciplined decision-making under constraints

Assignment: Weekly Standup Reflection

https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection

(Individual Assignment)
Due Feb 19 (and every week thereafter)

Assignment: The Overdose

https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose

(Individual Assignment)
Due Apr 23

Assignment: Requirements Report Document

https://rrostock1.github.io/Ursinus-CS375/Project/Requirements

(Group Assignment)
Due Feb 26

Due Next Week (Feb 26, by Midnight)

- Weekly Standup Reflection
- Software Requirements Document
- Git Homework Assignment